



BITS Pilani

Pilani | Dubai | Goa | Hyderabad

Artificial and Computational Intelligence

AIMLCZG557

ACI Team



AIMLCZG557 – CS#6

Ant Colony Optimization and Neural Architecture Search (NAS)

Agenda for CS #6

- 1) Recap of CS #5
- 2) Introduction to ACO
- 3) Neural Architecture Search
 - 1) Why?
 - 2) What?
 - 3) CoDeepNEAT
 - 4) Example
- 4) Q&A!

Ant Colony Optimization



Parameters used in ACO

Parameter	Description
N	Total No of ants ; $N > 1$
τ_o	Initial pheromone amount
τ_{ij}	Amount of pheromone deposited while traversing from i to j
η_{ij}	$1/(\text{Cost of link (i,j)})$
α	Importance coefficient of pheromone intensity
β	Importance coefficient of route cost
ρ	evaporation co-efficient; $0 < \rho < 1$
$visit_k$	Visited nodes table of k^{th} ant
Q	Importance - Constant value pertaining to pheromone trail
f_k	Route cost obtained by ant k

General pseudo-code

Procedure ACO

Schedule Activities

Initialization

Construction

Update Pheromone

Daemon Actions {optional}

// local search, elitism

End schedule activities

End ACO

ACO Steps



Initialization

- Place predefined number of ants on starting point
- Set values for parameters α, β, ρ .
- Set τ_0 to 0.

– Construction

- Compute the next node transition probability

$$NTP_{ij} = \frac{(\tau_{ij})^{\alpha} (\eta_{ij})^{\beta}}{\sum_{h \notin \text{visit}_k} (\tau_{ih})^{\alpha} (\eta_{ih})^{\beta}}$$

ACO Steps



Pheromone updation:

- Pheromone reinforcement & pheromone evaporation
- Direct impact on the exploitation (enhancing found food path) & exploration (discovering new path) of ant algorithms

$$\tau_{ij}^{new} = (1 - \rho)\tau_{ij}^{old} + \Delta\tau_{ij}^k$$

- Amount of pheromone deposited on (i,j) by kth at that timestamp is given by

$$\Delta\tau_{ij}^k = \begin{cases} \frac{Q}{f_k} & \text{if } k^{th} \text{ ant passes } i \text{ and } j \\ 0 & \text{otherwise} \end{cases}$$

- **Stopping criteria:** reaching predetermined number of iterations

Problem: Reaching pre-determined number of iterations before reaching destination leading to ant drop

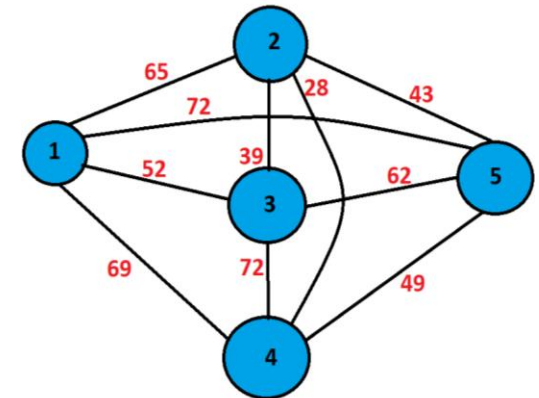
Travelling Salesman Problem



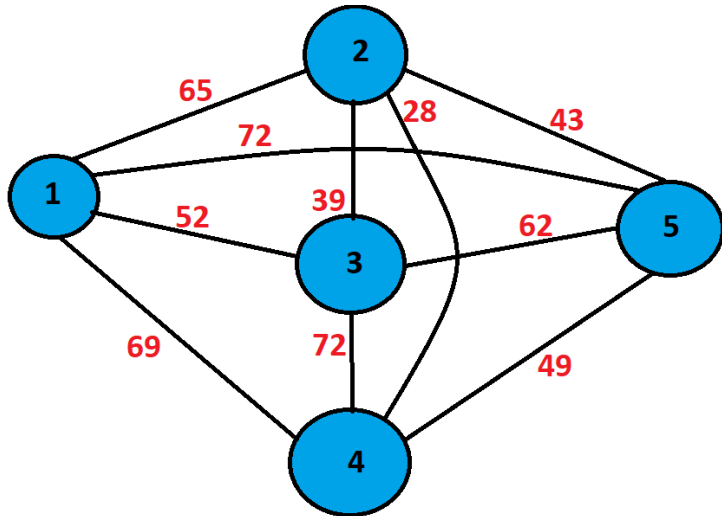
Problem: Given n cities, the goal is to find shortest path going through all cities and visiting each exactly once

- Consider a complete graph
- d_{ij} is the route cost over (i,j) $\{f_k\}$

- Each ant builds its own tour from starting city
- Each ant chooses a town to go to with a probability
- Keep tabs on visit list of each ant
- When tour completed, lay pheromone on each edge visited
- Next city j after city i chosen according to probability rule



Travelling Salesman Problem using ACO



No. of ants = No. of cities = 5
Start at 4

$$\alpha = 0.5$$

$$\beta = 0.75$$

$$\rho = 0.1$$

$$Q = 100$$

$$\tau_{12} = \tau_{21} = 0.54$$

$$\tau_{13} = \tau_{31} = 0.53$$

$$\tau_{14} = \tau_{41} = 0.35$$

$$\tau_{15} = \tau_{51} = 0.24$$

$$\tau_{23} = \tau_{32} = 0.53$$

$$\tau_{24} = \tau_{42} = 0.39$$

$$\tau_{25} = \tau_{52} = 0.18$$

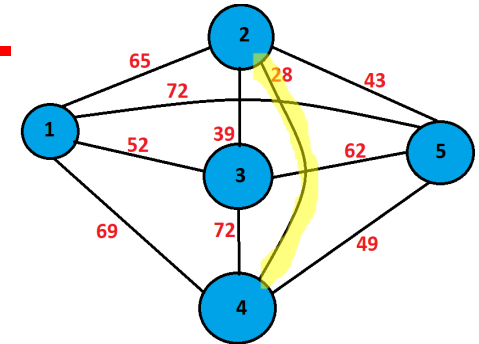
$$\tau_{34} = \tau_{43} = 0.32$$

$$\tau_{35} = \tau_{53} = 0.9$$

$$\tau_{45} = \tau_{54} = 0.68$$

	1	2	3	4	5
1	0	65	52	69	72
2	65	0	39	28	43
3	52	39	0	72	62
4	69	28	72	0	49
5	72	43	62	49	0

Step 1: Next Transition Probability



$$NTP_{ij} = \frac{(\tau_{ij})^\alpha (\eta_{ij})^\beta}{\sum_{n \in \text{visit}_k} (\tau_{in})^\alpha (\eta_{in})^\beta}$$

$$NTP_{41} = \frac{(\tau_{41})^\alpha (\eta_{41})^\beta}{(\tau_{41})^\alpha (\eta_{41})^\beta + (\tau_{42})^\alpha (\eta_{42})^\beta + (\tau_{43})^\alpha (\eta_{43})^\beta + (\tau_{45})^\alpha (\eta_{45})^\beta}$$

$$= \frac{(0.35)^{0.5} \cdot (1/69)^{0.75}}{(0.35)^{0.5} \cdot (1/69)^{0.75} + (0.39)^{0.5} \cdot (1/28)^{0.75} + (0.32)^{0.5} \cdot (1/72)^{0.75} + (0.68)^{0.5} \cdot (1/49)^{0.75}}$$

$$= \frac{(0.35)^{0.5} \cdot (0.0145)^{0.75}}{(0.35)^{0.5} \cdot (0.0145)^{0.75} + (0.39)^{0.5} \cdot (0.0357)^{0.75} + (0.32)^{0.5} \cdot (0.0139)^{0.75} + (0.68)^{0.5} \cdot (0.0204)^{0.75}}$$

$$= \frac{0.0247}{0.1434} \quad \boxed{P_{41} = 0.1723}$$

$$P_{42} = \frac{(0.39)^{0.5} \cdot (1/28)^{0.75}}{0.1434}$$

$$= \frac{0.0513}{0.1434}$$

$$P_{43} = \frac{(0.32)^{0.5} \cdot (\frac{1}{72})^{0.75}}{0.1434}$$

$$= \frac{0.0229}{0.1434}$$

$$P_{45} = \frac{(0.68)^{0.5} \cdot (\frac{1}{49})^{0.75}}{0.1434}$$

$$= \frac{0.0445}{0.1434}$$

P_{42} is max.
Therefore,
move from
4 to 2.

$$\boxed{P_{42} = 0.3577}$$

$$\boxed{P_{43} = 0.1596}$$

$$\boxed{P_{45} = 0.3104}$$

Pheromone Updation

$$\tau_{ij}^{\text{new}} = (1 - \rho)\tau_{ij}^{\text{old}} + \Delta\tau_{ij}^k$$

$$\Delta\tau_{ij}^k = \begin{cases} \frac{Q}{L_k} & \text{if ant } k \text{ passes edge } ij \\ 0 & \text{otherwise} \end{cases}$$

Since ant moved from 4 to 2,
calculate $\Delta\tau_{12}$:

$$\tau_{12} = (1 - 0.1)(0.54) + 0 = 0.486$$

$$\tau_{13} = (1 - 0.1)(0.53) + 0 = 0.477$$

$$\tau_{14} = (1 - 0.1)(0.35) + 0 = 0.315$$

$$\tau_{15} = (1 - 0.1)(0.24) + 0 = 0.216$$

$$\tau_{23} = (1 - 0.1)(0.53) + 0 = 0.477$$

$$\tau_{24} = \tau_{42} = (1 - 0.1)(0.39) + \frac{100}{28} = 3.924$$

$$\tau_{25} = (1 - 0.1)(0.16) + 0 = 0.162$$

$$\tau_{34} = (1 - 0.1)(0.32) + 0 = 0.288$$

$$\tau_{35} = (1 - 0.1)(0.9) + 0 = 0.81$$

$$\tau_{45} = (1 - 0.1)(0.68) + 0 = 0.612$$

Step 2: Next Transition Probability



$$\text{Visit}_k = 4 - 2$$

Find P_{21} , P_{23} and P_{25}

$$P_{21} = \frac{(0.486)^{0.5} \left(\frac{1}{65}\right)^{0.75}}{(0.486)^{0.5} \left(\frac{1}{65}\right)^{0.75} + (0.477)^{0.5} \left(\frac{1}{39}\right)^{0.75} + (0.162)^{0.5} \left(\frac{1}{43}\right)^{0.75}}$$

$$= \frac{(0.6971)(0.0437)}{(0.6971)(0.0437) + (0.6907)(0.064) + (0.4025)(0.0596)}$$

$$P_{21} = \frac{0.0305}{0.0987}$$

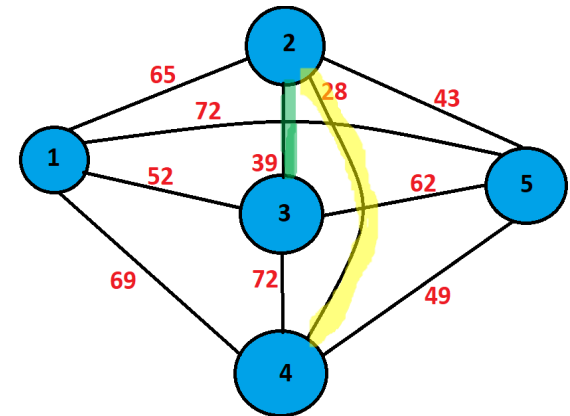
$$P_{21} = 0.3086$$

$$P_{23} = \frac{0.0443}{0.0987}$$

$$P_{23} = 0.4485$$

$$P_{25} = \frac{0.0240}{0.0987}$$

$$P_{25} = 0.2429$$



P_{23} is maximum. Now ant moves from 2 to 3.

Pheromone Updation

$$\tau_{ij}^{\text{new}} = (1 - \rho)\tau_{ij}^{\text{old}} + \Delta\tau_{ij}^k$$

$$\Delta\tau_{ij}^k = \begin{cases} \frac{Q}{L_k} & \text{if ant } k \text{ passes edge } ij \\ 0 & \text{otherwise} \end{cases}$$

Since ant moved from 2 to 3,
calculate $\Delta\tau_{23}$:

$$\tau_{12} = (1 - 0.1) \times 0.486 + 0 = 0.4374$$

$$\tau_{13} = (1 - 0.1) \times 0.477 + 0 = 0.4293$$

$$\tau_{14} = (1 - 0.1) \times 0.315 + 0 = 0.2835$$

$$\tau_{15} = (1 - 0.1) \times 0.216 + 0 = 0.1944$$

$$\tau_{23} = (1 - 0.1) \times 0.477 + \frac{100}{39} = 2.9934$$

$$\tau_{24} = (1 - 0.1) \times 3.9224 + 0 = 3.5302$$

$$\tau_{25} = (1 - 0.1) \times 0.162 + 0 = 0.1458$$

$$\tau_{34} = (1 - 0.1) \times 0.288 + 0 = 0.2592$$

$$\tau_{35} = (1 - 0.1) \times 0.810 + 0 = 0.729$$

$$\tau_{45} = (1 - 0.1) \times 0.612 + 0 = 0.5508$$

Step 3: Next Transition Probability



$$\text{Visit}_k = 4 - 2 - 3$$

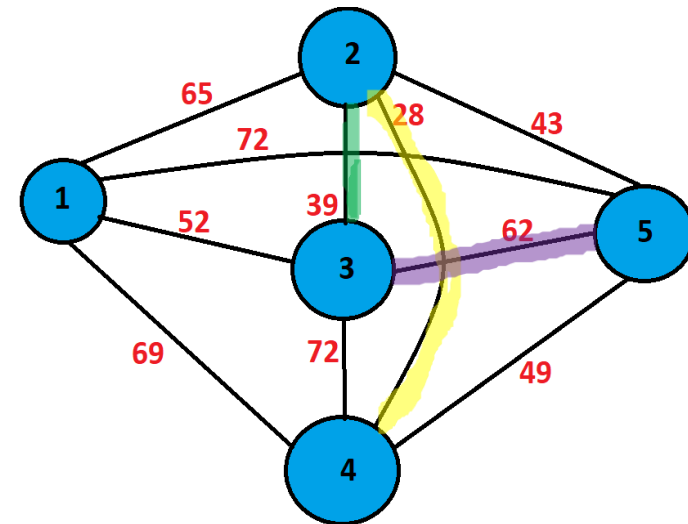
Find P_{31} and P_{35}

$$P_{31} = \frac{(0.4293)^{0.5} \left(\frac{1}{52}\right)^{0.75}}{(0.4293)^{0.5} \left(\frac{1}{52}\right)^{0.75} + (0.729)^{0.5} \left(\frac{1}{62}\right)^{0.75}}$$
$$= \frac{0.0338}{0.0725}$$

$$P_{31} = 0.4668$$

$$P_{35} = \frac{0.0386}{0.0725}$$

$$P_{35} = 0.5332$$



P_{35} is maximum. Now ant moves from 3 to 5.

Pheromone Updation

$$\tau_{ij}^{\text{new}} = (1 - \rho)\tau_{ij}^{\text{old}} + \Delta\tau_{ij}^k$$

$$\Delta\tau_{ij}^k = \begin{cases} \frac{Q}{L_k} & \text{if ant } k \text{ passes edge } ij \\ 0 & \text{otherwise} \end{cases}$$

Since ant moved from 3 to 5,
calculate $\Delta\tau_{35}$:

$$\tau_{12} = (1 - 0.1) \times 0.4374 + 0 = 0.3937$$

$$\tau_{13} = (1 - 0.1) \times 0.4293 + 0 = 0.3864$$

$$\tau_{14} = (1 - 0.1) \times 0.2835 + 0 = 0.2552$$

$$\tau_{15} = (1 - 0.1) \times 0.1944 + 0 = 0.1750$$

$$\tau_{23} = (1 - 0.1) \times 2.9334 + 0 = 2.6941$$

$$\tau_{24} = (1 - 0.1) \times 3.5302 + 0 = 3.1772$$

$$\tau_{25} = (1 - 0.1) \times 0.1458 + 0 = 0.1312$$

$$\tau_{34} = (1 - 0.1) \times 0.2592 + 0 = 0.2333$$

$$\tau_{35} = (1 - 0.1) \times 0.729 + \frac{100}{62} = 2.269$$

$$\tau_{45} = (1 - 0.1) \times 0.5508 + 0 = 0.4957$$

Step 4: Next Transition Probability



Visit_k = 4 - 2 - 3 - 5 - 1

Now ant moves from 5 to 1.

$$\tau_{12} = 0.9 \times 0.3937 + 0 = 0.3543$$

$$\tau_{13} = (1 - 0.1) \times 0.3864 + 0 = 0.3477$$

$$\tau_{14} = (1 - 0.1) \times 0.2552 + 0 = 0.2296$$

$$\tau_{15} = (1 - 0.1) \times 0.175 + \frac{100}{72} = 1.5464$$

$$\tau_{23} = (1 - 0.1) \times 2.6941 + 0 = 2.4247$$

$$\tau_{24} = (1 - 0.1) \times 3.1772 + 0 = 2.8595$$

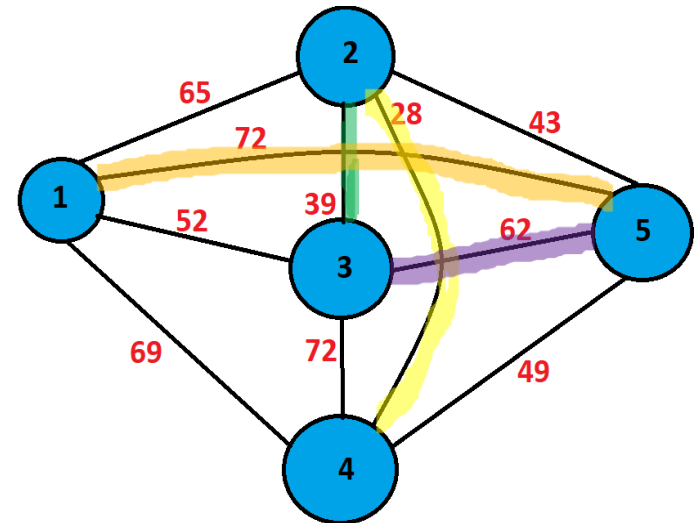
$$\tau_{25} = (1 - 0.1) \times 0.1312 + 0 = 0.1181$$

$$\tau_{34} = (1 - 0.1) \times 0.2333 + 0 = 0.21$$

$$\tau_{35} = (1 - 0.1) \times 2.269 + 0 = 2.0421$$

$$\tau_{45} = (1 - 0.1) \times 0.4957 + 0 = 0.4461$$

Pheromone Updation:



Step 5: Next Transition Probability

$$\text{Visit}_k = 4 - 2 - 3 - 5 - 1 - 4$$

Now ant moves from 1 to 4 (origin).

$$\tau_{12} = (1 - 0.1) \times 0.3543 + 0 = 0.3189$$

$$\tau_{13} = (1 - 0.1) \times 0.3477 + 0 = 0.313$$

$$\tau_{14} = (1 - 0.1) \times 0.2296 + \frac{100}{69} = 1.6559$$

$$\tau_{15} = (1 - 0.1) \times 1.5464 + 0 = 1.3917$$

$$\tau_{23} = (1 - 0.1) \times 2.4247 + 0 = 2.1822$$

$$\tau_{24} = (1 - 0.1) \times 2.8595 + 0 = 2.5735$$

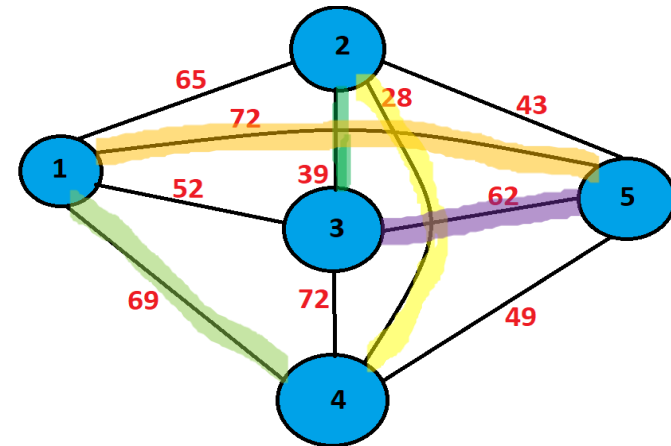
$$\tau_{25} = (1 - 0.1) \times 0.1181 + 0 = 0.1063$$

$$\tau_{34} = (1 - 0.1) \times 0.21 + 0 = 0.189$$

$$\tau_{35} = (1 - 0.1) \times 2.0421 + 0 = 1.8379$$

$$\tau_{45} = (1 - 0.1) \times 0.4461 + 0 = 0.4015$$

Pheromone Updation:



Module 3 : Problem Solving Agent using Search

- A. Informed Search Algorithms ☒
- B. Heuristic Functions ☒
- C. Local Search Algorithms & Optimization Problems ☒
- D. Neural Architecture Search**



The Era of Self-Designing AI

- CoDeepNEAT (Co-evolving Deep NeuroEvolution of Augmenting Topologies) is an *automated* method for designing deep neural network architectures.
 - Traditional AI relies on human experts to manually design architectures; CoDeepNEAT replaces this with evolutionary search.
 - Inspired by natural evolution, architectures are gradually improved through selection, mutation, and recombination.
 - Goal: Automatically evolve a neural network that can accurately classify images such as “Cat” vs “Dog” without manual design decisions.
-



The Human Bottleneck

- Designing deep neural networks manually is challenging due to a vast search space of possible architectures and hyperparameters.
 - Human experts rely on trial-and-error and prior knowledge, which limits exploration of novel architectures.
 - As network complexity increases, manual optimization becomes inefficient and impractical.
 - CoDeepNEAT addresses this by evolving a population of architectures and selecting the most effective ones based on performance.
-



DNA vs. Weights (The Hybrid Optimization Approach)

CoDeepNEAT follows a bilevel optimization strategy combining evolution and learning.

1. Architecture (DNA / Topology): Designed using evolutionary algorithms (high-level search).
 2. Weights (Learned Parameters): Optimized using gradient descent during training (low-level learning).
 - Evolution explores structural design, while learning optimizes performance within that structure.
 - This synergy enables automatic discovery of architectures that are both effective and efficient.
-



What is Neuroevolution?

Neuroevolution = Using genetic algorithms to evolve neural networks

How It Works (Core Loop)

1. Create many neural networks (population)
2. Train each network
3. Measure performance (fitness)
4. Select best networks
5. Generate new networks using:
 - Mutation
 - Crossover
6. Repeat

- **Population** → Set of networks
- **Fitness** → How good the network performs
- **Generation** → One cycle of evolution

Understanding NEAT



- NEAT stands for NeuroEvolution of Augmenting Topologies.
- It is a method that evolves neural networks automatically instead of designing them manually.
- A neural network is represented as a graph. Nodes represent neurons and edges represent connections between them.
- It starts with very simple networks having minimal connections. Over generations, it gradually adds new nodes and connections to increase complexity.
- Each new connection is assigned a unique innovation number. These numbers help match corresponding genes correctly during crossover.
- Similar networks are grouped into species based on their structure. Each species evolves separately, maintaining diversity in the population.



Limitation of NEAT

- NEAT performs well for small and simple neural networks but is not effective for deep learning models with many layers.
 - Modern AI systems use deep architectures such as CNNs and LSTMs, which require layer-wise design and complex structures.
 - NEAT evolves individual neurons and connections, but not full layered architectures.
 - Therefore, there is a need for methods that can evolve deep neural networks.
 - This limitation led to advanced approaches like DeepNEAT and CoDeepNEAT.
-

DeepNEAT

- DeepNEAT extends NEAT to evolve deep neural networks with complex, multi-layer architectures.
 - In NEAT, nodes represent individual neurons, whereas in DeepNEAT, each node represents an entire layer.
 - Each node stores layer details, including the type (Convolution, Dense, LSTM) and hyperparameters such as number of filters, kernel size, and activation function.
 - Edges define how layers are connected and represent the flow of data between them.
 - Unlike NEAT, edges in DeepNEAT do not store weights.
-



How DeepNEAT Builds a Network

- The process starts with a chromosome represented as a graph, which is traversed step by step.
 - For each node, a corresponding neural network layer is created, and all layers are connected based on the graph structure.
 - When multiple inputs occur, they are combined using concatenation or summation operations.
 - If there is a size mismatch between layers, pooling or downsampling techniques are applied.
 - The final output is a complete deep neural network built from the evolved structure.
-

Problem with DeepNEAT

- Networks generated by DeepNEAT tend to be random, complex, and lack clear structure.
 - In contrast, modern models like ResNet and GoogLeNet use repeated and well-organized blocks with modular design.
 - DeepNEAT does not reuse learned structures and fails to create repeatable patterns.
 - Therefore, there is a need for reusable building blocks to design structured and efficient networks.
-



CoDeepNEAT

- CoDeepNEAT improves DeepNEAT by separating evolution into two parts, enabling better structure and reuse.
 - The blueprint defines the overall network structure and acts as a skeleton for organizing components.
 - The module is a small neural network that serves as a reusable building block.
 - This approach reflects real-world deep networks, which reuse blocks instead of building everything from scratch.
 - As a result, networks become more structured, efficient, and scalable.
-

Genotype Representation in CoDeepNEAT

Genotype Representation

- Composed of two co-evolving populations:
 - **Blueprints** – define the overall network structure
 - **Modules** – define reusable neural sub-networks
 - Blueprint nodes represent module species (placeholders), not actual layers
 - Modules contain layer types and hyperparameters (e.g., Convolution, Dense, LSTM)
-

Genotype Representation in CoDeepNEAT

Blueprint (Macro-Architecture)

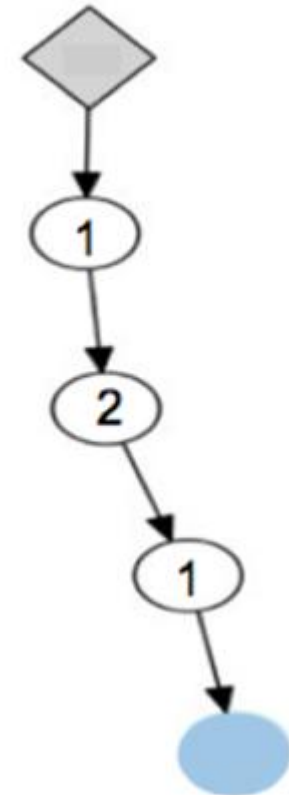
Represents the high-level structure of the network

Each node:

- ❖ Refers to a module species ID (placeholder)
- ❖ Does NOT represent a direct layer

Edges define data flow between modules

Blueprint



Genotype Representation in CoDeepNEAT

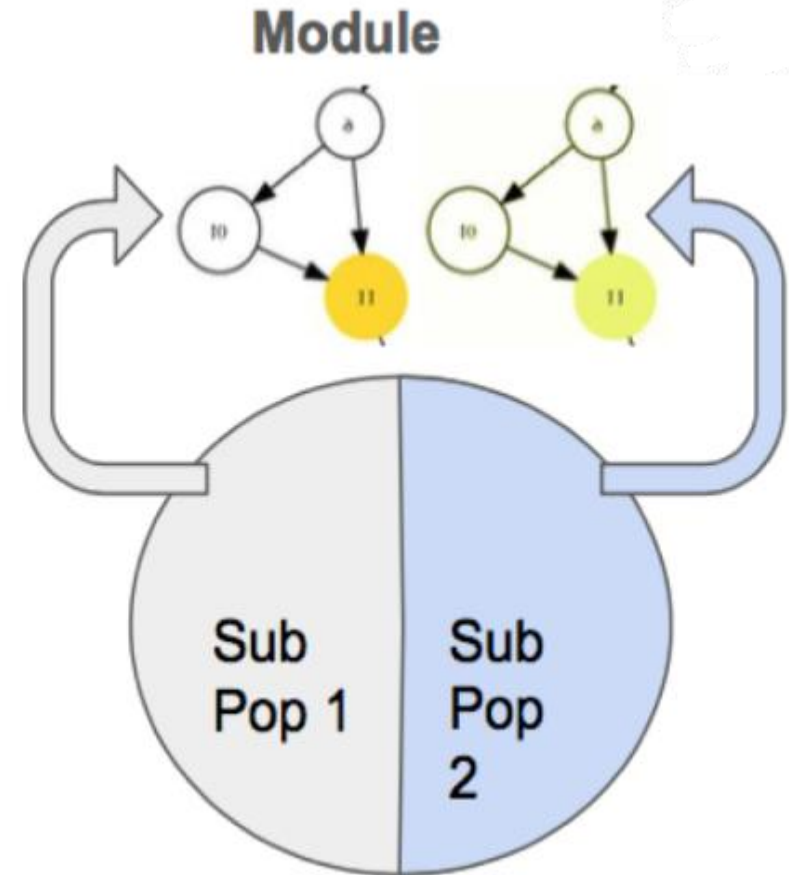
Modules (Micro-Architecture)

Small neural sub-networks (building blocks)

Each module defines:

- ❖ Layer types (Conv, Dense, LSTM)
- ❖ Hyperparameters (filters, neurons, activation)

Modules are evolved independently in subpopulations



Phenotype Construction & Network Assembly

Phenotype Construction

The phenotype is the actual neural network used for training

Built by:

- Replacing each **blueprint node with a selected module**
- Connecting modules based on blueprint edges

Assembly Process

1. Select a blueprint
 2. For each node:
 - ❖ Choose a module from its species
 3. Replace nodes with modules
 4. Connect them → forms a complete deep network
-



Phenotype Construction & Network Assembly

Innovation Numbers

Unique IDs assigned to structural mutations

Ensure:

- ❖ Correct alignment during crossover
- ❖ Meaningful recombination of architectures

Resulting Network

Produces:

- ❖ Deep architectures
 - ❖ Reusable patterns
 - ❖ Efficient search space exploration
-

Phenotype Construction – Concept & Process

What is Phenotype Construction?

- Phenotype = *final* deep neural network
- Built from genotype (blueprint + modules)
- Used for training and fitness evaluation

Key Idea

- Small modules are assembled like building blocks
 - Creates a large, deep neural network automatically
-

Phenotype Construction – Concept & Process

1. Select a Blueprint

- Defines overall structure (flow of data)

2. Identify Nodes in Blueprint

- Each node = module placeholder (species ID)

3. Select Modules

- For each node, pick a module from its subpopulation

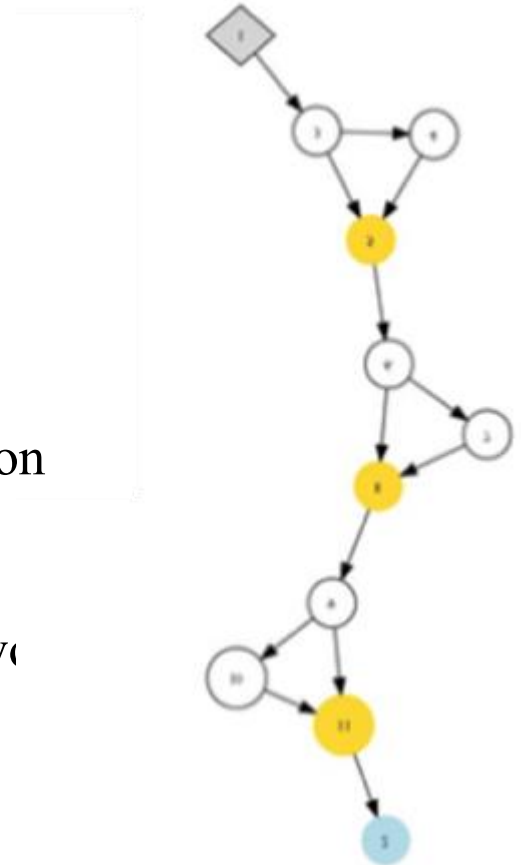
4. Replace Nodes with Modules

- Each node is expanded into a small neural subnetwork

5. Connect Modules

- Connections follow blueprint topology

Assembled Network



Phenotype Construction – Concept & Process

◆ Top Node (Input)

- Starting point of the network

● White Nodes

- Represent intermediate connections and flow

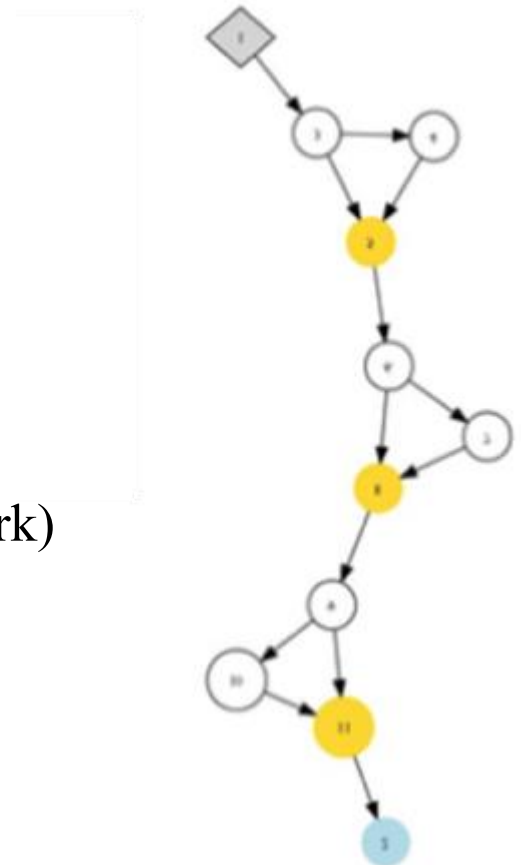
● Yellow Nodes (Important)

- These indicate module insertions
- Each yellow node = expanded module (subnetwork)

● Bottom Node (Output)

- Final prediction/output layer

Assembled Network

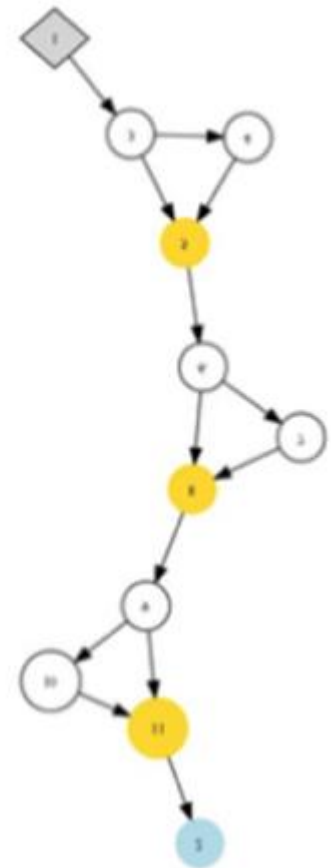


Phenotype Construction – Concept & Process

What is Happening Here?

- Each yellow node was originally a single blueprint node
- It is now replaced by a complex module
- These modules are:
 - ❖ Reused
 - ❖ Repeated at different positions
- Connections between them follow the blueprint structure

Assembled Network





Initialization – The Big Bang

- Evolution begins with a population of randomly generated, simple neural network architectures.
 - Example: Initialize 100 networks with minimal layers and basic connectivity.
 - These initial networks act as the starting point for evolutionary search.
 - Starting with simple structures allows gradual complexity growth through mutations and crossover.
 - This approach promotes efficiency and avoids unnecessary complexity in early stages.
-

The Evolutionary Mechanics

Fitness Evaluation Phase



- Each candidate architecture is instantiated as a neural network and trained using gradient descent.
 - Example: A CNN architecture is trained on a cat vs dog dataset containing 10,000 labeled images.
 - Training is performed for a limited number of epochs (e.g., 5–10 epochs) to evaluate learning capability.
 - The objective is to identify architectures that achieve higher accuracy within limited training time.
-



Defining Fitness (Performance Metric)

- Fitness is computed as classification accuracy on a validation dataset.
 - Example: If the network correctly classifies 90 out of 100 validation images, $\text{fitness} = 90\%$.
 - This ensures evaluation is based on generalization, not memorization.
 - Higher fitness values indicate better architectural suitability for the task.
-



- Parent architectures are selected using a probabilistic approach based on fitness values.
- Example:
 - Network A (90% accuracy) → higher selection probability
 - Network B (50% accuracy) → lower selection probability
- Selection ensures that better-performing architectures are more likely to reproduce.

Detailed Selection Example

- Consider four architectures with fitness values:
 - A: 80%, B: 60%, C: 40%, D: 20%
 - Selection probabilities are proportional to these values.
 - Example outcome: A and B are selected multiple times; C occasionally; D rarely.
 - This maintains a balance between exploitation and exploration.
-

Crossover – Architectural Recombination

- Two selected parent architectures are combined to produce a new architecture.
 - Example:
 - Parent 1: Convolution → Pooling → Dense
 - Parent 2: Convolution → Convolution → Dense
 - Child: Convolution → Convolution → Dense (inherits deeper feature extraction capability)
-



Innovation Numbers in Crossover

- Each structural component is assigned an innovation number.
 - During crossover, matching components are aligned based on these identifiers.
 - Example:
 - Both parents contain a convolution layer with similar function → aligned
 - Extra layer in one parent → inherited if beneficial
 - This ensures valid and meaningful architecture generation.
-

Mutation

Structural and Parametric Variation



- Mutation introduces changes to explore new architectures.
- Types of mutation:
 - Add a new layer (e.g., additional convolution layer)
 - Add a connection (skip connection between layers)
 - Modify hyperparameters (e.g., filter size, activation function)
- Example: Changing filter size from 3×3 to 5×5 to capture broader features.

Detailed Mutation Example

- Original architecture: Convolution (3×3) $\rightarrow$ Pooling $\rightarrow$ Dense
 - Mutation: Convolution filter size changed to 5×5
 - Result: Improved ability to capture larger patterns such as full face structures
 - This may increase validation accuracy from 85% to 92%.
-

Speciation – Structural Grouping

- Architectures are grouped into species based on similarity metrics.
 - Example:
 - Species 1: shallow networks (1–2 layers)
 - Species 2: deeper networks (3–5 layers)
 - Competition occurs within species rather than globally.
 - This preserves diversity and allows new structures to evolve.
-



The Generational Evolution Process

- The evolutionary loop is repeated across multiple generations.
 - Each generation includes evaluation, selection, crossover, mutation, and speciation.
 - Example progression (cat vs dog classification):
 - Generation 1: ~50% accuracy (random performance)
 - Generation 20: ~75% accuracy (basic feature learning)
 - Generation 50: ~95% accuracy (advanced feature extraction)
 - Final architecture is selected based on highest validation performance.
-

Step 1: Input Image → Numerical Form

- Suppose we input a cat image (32×32 pixels)
 - Converted into a tensor:
 - $32 \times 32 \times 3 = 3072$ values
 - Example:
 - Pixel (R,G,B) = (120, 80, 60) → normalized → (0.47, 0.31, 0.23)
 - At this stage, the system does NOT know it is a cat - only numbers.
-

Step 2: Evolved Architecture is Selected

- CoDeepNEAT has already evolved a network like:
 - Module 1: Conv (3×3) + ReLU
 - Module 2: Conv (5×5) + Pool
 - Module 3: Dense layer
 - This structure was not manually designed; It was discovered through evolution.
-

Step 3: Forward Pass Begins (Feature Extraction)

Layer 1: Convolution (Edge Detection)

- Apply filter (3×3 kernel)
- Example output:
 - Detects edges of ears, body outline
- Output: Feature map highlighting edges

Layer 2: Activation (ReLU)

- Formula:

$$f(x) = \max(0, x)$$

- Removes negative values
 - Keeps only important signals
-



Step 3: Forward Pass Begins (Feature Extraction)

Layer 3: Convolution (Texture Detection)

Combines edges → detects patterns

Example:

- Fur texture
- Eye regions

Now the network starts distinguishing cat-like texture

Layer 4: Pooling (Dimension Reduction)

Example: Max pooling (2×2)

Reduces size but keeps strongest features

Focus: “Is feature present?” not “Where exactly?”



Step 3: Forward Pass Begins (Feature Extraction)

Layer 5: Higher-Level Features

Combines patterns into objects:

- Triangular ears
- Small nose
- Whisker lines

These are cat-specific features



Step 4: Fully Connected Layer (Decision Preparation)

All extracted features are flattened into a vector

Example:

$[0.2, 0.8, 0.1, 0.9, \dots]$

This layer learns relationships like:

“If whiskers + triangular ears $\rightarrow$ likely cat”

Step 5: Output Layer (Softmax Classification)

$$P(y_i) = \frac{e^{z_i}}{\sum e^{z_j}}$$

Suppose output:

- Cat = 0.92
- Dog = 0.08

Final Prediction:

Cat (92% confidence)

Step 6: During Training (Backpropagation)

If prediction is wrong:

Example:

- True label: Dog
- Predicted: Cat

Error is calculated:

$$\text{Loss} = \text{Predicted} - \text{Actual}$$

Weights are updated using gradient descent

Step 7: Where CoDeepNEAT Helps

Instead of manually designing:

- Number of layers
- Filter sizes
- Connections

CoDeepNEAT evolves:

- Best combination of modules
 - Best topology
 - Best hyperparameters
-



Step 8: Real Example of Evolution Impact

During evolution:

Network A:

- Only detects edges → Accuracy: 60%

Network B:

- Detects fur + ears → Accuracy: 85%

Network C (after mutation):

- Adds skip connection → Accuracy: 92%

Evolution selects Network C as the best

Step 9: Final Model Behavior

When deployed:

1. Input image → converted to numbers
 2. Passed through evolved architecture
 3. Features extracted automatically
 4. Output → Cat or Dog
-

Step 10: Key Insight

The system does not “see” animals like humans

It detects:

➤ Edges → textures → parts → object

CoDeepNEAT ensures:

➤ The best architecture is discovered automatically



Modules & Blueprints – Scaling in CoDeepNEAT

- Evolving an entire deep neural network as a single structure is computationally expensive and difficult to optimize.
- CoDeepNEAT addresses this by decomposing the architecture into two evolving populations:
 - Modules: reusable subnetwork components
 - Blueprints: high-level structural graphs
- **Example:** Instead of evolving one full CNN for cat vs dog classification, smaller subnetworks (feature extractors) and their arrangement are evolved separately.
- This decomposition enables efficient exploration of deeper and more complex architectures.



Modules – Reusable Subnetwork Components

- A module represents a small neural network (e.g., a sequence of convolution, pooling, and activation layers).
 - Each module has its own topology and hyperparameters that evolve over generations.
 - Example (Cat vs Dog):
 - Module A: Convolution (3×3) $\rightarrow$ ReLU $\rightarrow$ Max Pooling (detects edges and textures)
 - Module B: Convolution (5×5) $\rightarrow$ ReLU (captures larger patterns like face regions)
 - These modules act as feature extractors and can be reused multiple times within a network.
-



Blueprints – High-Level Network Structure

- A blueprint is a graph where each node points to a module species instead of a specific layer.
 - It defines how modules are arranged and connected to form a complete architecture.
 - Example (Cat vs Dog):
 - – Node 1 → Module A (edge detector)
 - Node 2 → Module B (pattern detector)
 - Node 3 → Fully connected layer
 - The blueprint determines the overall flow of data and interaction between modules.
-

Assembly Process

Constructing the Neural Network



- Step 1: Select a blueprint from the blueprint population.
- Step 2: For each node in the blueprint, randomly select a module from the corresponding module species.
- Step 3: Replace blueprint nodes with selected modules and connect them as defined.
- Example:
 - Blueprint structure: [Module A $\rightarrow$ Module B $\rightarrow$ Output Layer]
 - Selected modules are inserted to form a full CNN
- The assembled network is then trained and evaluated on the cat vs dog dataset.

Shared Fitness – Coevolution Mechanism



- Fitness is assigned based on the performance of the assembled network.
- The fitness score is distributed back to both:
 - The blueprint used to assemble the network
 - All modules that participated in that network
- Example:
 - Assembled network achieves 92% accuracy on validation data
 - The blueprint and all contributing modules receive this fitness contribution
- This enables simultaneous improvement of module quality and blueprint design across generations.

Example

A company wants to automatically generate captions for uploaded images using .The system evolves:

- CNN modules for image feature extraction
- LSTM modules for text generation
- Blueprints for arranging modules

Consider the following modules:

Module ID	Structure
M1	CNN Feature Extractor
M2	LSTM Layer
M3	Dense + Softmax

Blueprint: $B1=[M1 \rightarrow M2 \rightarrow M3]$

Validation scores:

Network	BLEU Score
N1	0.71
N2	0.88
N3	0.64

Demonstrate the first iteration of:

1. Phenotype Construction
2. Fitness Evaluation
3. Selection
4. Crossover
5. Mutation



Step 1: Initialization

Generate initial blueprint and module populations.

Blueprint Population

$B1 = [M1 \rightarrow M2 \rightarrow M3]$

$B2 = [M1 \rightarrow M1 \rightarrow M3]$

Module Population

Module ID	Structure
M1	CNN Feature Extractor
M2	LSTM Layer
M3	Dense + Softmax



Step 2: Genotype Representation

Blueprint

Defines the overall network structure.

$$B1 = [M1 \rightarrow M2 \rightarrow M3]$$

Modules

Reusable subnetworks.

$M1 \rightarrow$ CNN Feature Extractor

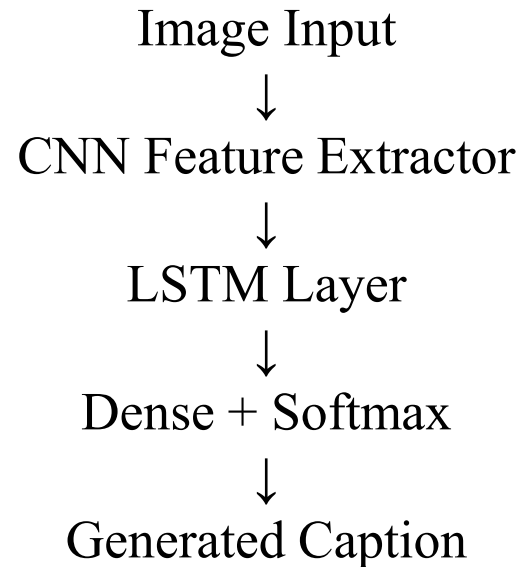
$M2 \rightarrow$ LSTM Layer

$M3 \rightarrow$ Dense + Softmax

Step 3: Phenotype Construction

Replace blueprint nodes with selected modules.

Constructed Architecture



This forms the final caption generation network.

Step 4: Fitness Evaluation

Fitness is computed using BLEU score.

Network	BLEU Score	Fitness
N1	0.71	0.71
N2	0.88	0.88
N3	0.64	0.64

Best Network:

$$N2 = 0.88$$

Hence N2 is selected as the strongest architecture.

Step 5: Selection

Using fitness proportionate selection:

(In fitness proportionate selection (roulette wheel selection):

- Higher fitness $\rightarrow$ higher probability of selection
- Lower fitness $\rightarrow$ lower probability)

Total Fitness:

$$0.71 + 0.88 + 0.64 = 2.23$$

$$\text{Probability of N1} = P(N1) = 0.71 / 2.23 = 0.318$$

$$\text{Probability of N2} = P(N2) = 0.88 / 2.23 = 0.394$$

$$\text{Probability of N3} = P(N3) = 0.64 / 2.23 = 0.287$$

Since:

- N2 has the highest probability
 - N1 has the second highest probability,
- they are selected as parent architectures for crossover.

Step 6: Crossover

Parent 1:

CNN $\rightarrow$ LSTM $\rightarrow$ Dense

Parent 2:

CNN $\rightarrow$ CNN $\rightarrow$ Dense

(Assume one-point crossover after the first module.)

Child 1

CNN $\rightarrow$ CNN $\rightarrow$ Dense

Child 2

CNN $\rightarrow$ LSTM $\rightarrow$ Dense

Step 7: Mutation

Mutation is applied to offspring architectures generated during crossover.

Before Mutation

Child 1: CNN $\rightarrow$ CNN $\rightarrow$ Dense

Child 2: CNN $\rightarrow$ LSTM $\rightarrow$ Dense

After Mutation

Child 1: CNN(3×3) $\rightarrow$ CNN(5×5) $\rightarrow$ Dense

Effect

- Captures larger image patterns
- Improves feature extraction capability

Child 2: CNN $\rightarrow$ LSTM(256) $\rightarrow$ Dense

Effect

- Improves sequence learning
- Better caption generation capability

Step 8: Add Mutated Offspring to Next Generation Population

The mutated children become part of the new population.

Example

Network	Architecture
Parent N1	CNN $\rightarrow$ LSTM $\rightarrow$ Dense
Parent N2	CNN $\rightarrow$ CNN $\rightarrow$ Dense
Mutated Child 1	CNN(3×3) $\rightarrow$ CNN(5×5) $\rightarrow$ Dense
Mutated Child 2	CNN $\rightarrow$ LSTM(256) $\rightarrow$ Dense

These architectures form the next generation population.

Step 9: Fitness Evaluation Again

All networks are trained and evaluated again.

Example

Network	Accuracy
Parent N1	88%
Parent N2	84%
Mutated Child 1	91%
Mutated Child 2	94%

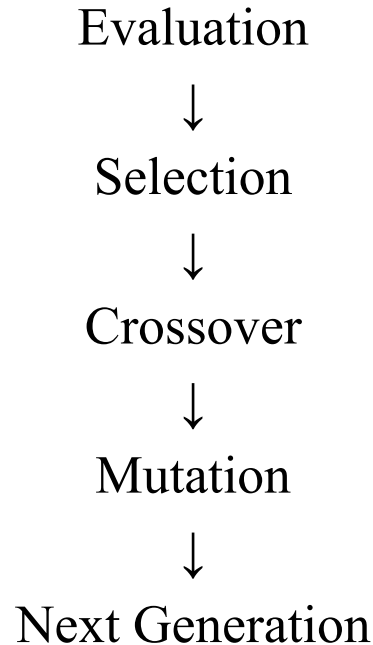
Now

Mutated Child 2

becomes the best architecture.

Step 10: Evolution Continues

The process repeats:



Over many generations:

- weak architectures disappear
 - strong architectures survive
 - deeper and better networks evolve automatically
-

After several generations, CoDeepNEAT discovers



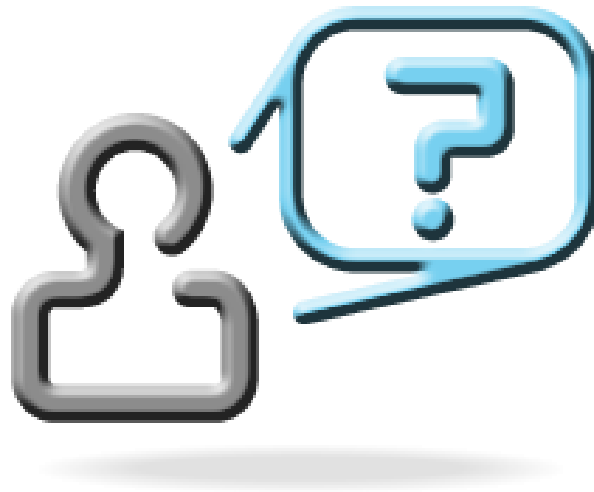
- optimal modules
- best blueprint structure
- efficient hyperparameters
- high-performing deep neural network architecture

This final network is deployed for tasks such as:

- image caption generation
- cat vs dog classification
- speech recognition
- medical image analysis.

Using CoDeepNEAT

- Modules and blueprints evolved separately
 - Better architectures were selected automatically using fitness evaluation
 - Two offspring architectures were generated through one-point crossover
 - Mutation modified module parameters and hyperparameters in offspring networks
 - Mutated offspring were added to the next generation population
 - Final evolved networks improved feature extraction and sequence learning capability
 - More accurate image captions were generated automatically
-



We will learn game play in the next class ...

Exercise 1

A warehouse robot begins at C1, tasked with collecting Items (It) and reaching the Dock (DV) at C4 on a 4x4 grid. The robot must navigate hazards like blocked cells (#), Spills (SP) with high movement costs, while leveraging Energy packs (E) to reduce costs. Using the A* algorithm, the robot calculates the optimal path by considering both actual path cost and a heuristic. The agent has four distinct actions: Move Up, Move Down, Move Left, and Move Right. However, it cannot move into blocked cells. The warehouse is represented as a 4×4 grid below:

	1	2	3	4
A	It	E		SP
B	SP	It	E	SP
C	S	E	It	DV
D	It	SP	SP	E

Entering SP: +12; collecting It: −8 (reward); energy pack E: +1.
Heuristic $h(n) = \text{Manhattan}(\text{robot}, \text{DV}) + (\# \text{remaining Items}) + (\# \text{adjacent SP to robot in resultant state}).$

- Expand and depict the search tree for exactly 2 levels. (Level 0 = Start).
- Calculate Path costs and heuristics (g/h/f) for all generated nodes.
- Apply A* search and show all the steps.

Exercise 2

1. Given two parent chromosomes:
 - Parent 1: [1, 2, 3, 4, 5, 6, 7, 8]
Parent 2: [8, 7, 6, 5, 4, 3, 2, 1]
 - Using single-point crossover after the 4th position, generate two children.
2. Suppose the fitness values of four individuals are:

Individual	Fitness
A	10
B	30
C	40
D	20

Which individual has the highest chance of being selected? Why?

ACI: Disclaimer and Acknowledgements

- Few content for these slides may have been obtained from prescribed books and various other source(s) on the Internet.
- We hereby acknowledge all the contributors for their material and inputs and gratefully acknowledge people others who made their course materials freely available online. We have provided source information wherever necessary.
- We have added and modified the content to suit the requirements of the class dynamics & live session's lecture delivery flow for presentation.
- This *is not a full-fledged* reading materials. Students are requested to refer to the textbook w.r.t detailed content as per the course handout as shared over e-learning portal - Taxila.
- **Slide Source / Preparation / Review:**
- From BITS Pilani WILP: Prof. Rajavadhana, Prof. Indumathi, Prof. Sangeetha and Prof Parthasarathy PD, Prof Ramya Devi
- From BITS On-campus & External : Mr.Santosh GSK

Thank You for your
time & attention !

Slides are Licensed Under : [CC BY-NC-SA 4.0](https://creativecommons.org/licenses/by-nc-sa/4.0/)

